

BTC•FUSION SYSTEM & MODEL ARCHITECTURE

How the pipeline is built, how a transaction is analysed end to end, what the models are, what they were trained on, and exactly where the machine learning sits.

CONTENTS

1	What the system is for	7	Training data and splits
2	System architecture	8	Attribution — network ⇔ chain
3	Data flow — the nine stages	9	Explainability
4	How one transaction is analysed	10	Measured results
5	Where the AI is	11	Limits and failure modes
6	Model specifications		

Every figure in this document is read from a build artefact, not quoted from memory. Unless stated otherwise the profile is **bulk** — 2,467,299 rows at a realistic 0.39% positive rate — and the source is `artifacts/bulk/manifest.json`, regenerated by `make reproduce`. Model `feature_version 7`, seed `20260826`, git `39e280a`, backend `sklearn_histgb`, trained 2026-08-28.

1 · WHAT THE SYSTEM IS FOR

Commercial chain-analysis tools are a purchasable, solved product. None of them ingest packet capture, because a private company cannot lawfully tap an ISP. That capability gap is the reason this problem statement exists.

Every blockchain tool can tell you that money moved suspiciously. This one correlates the **network layer** — source IP, port, arrival timestamp — with the **chain layer** — TXID, addresses, amounts — to say *who moved it*, and refuses to answer when the evidence is shared infrastructure.

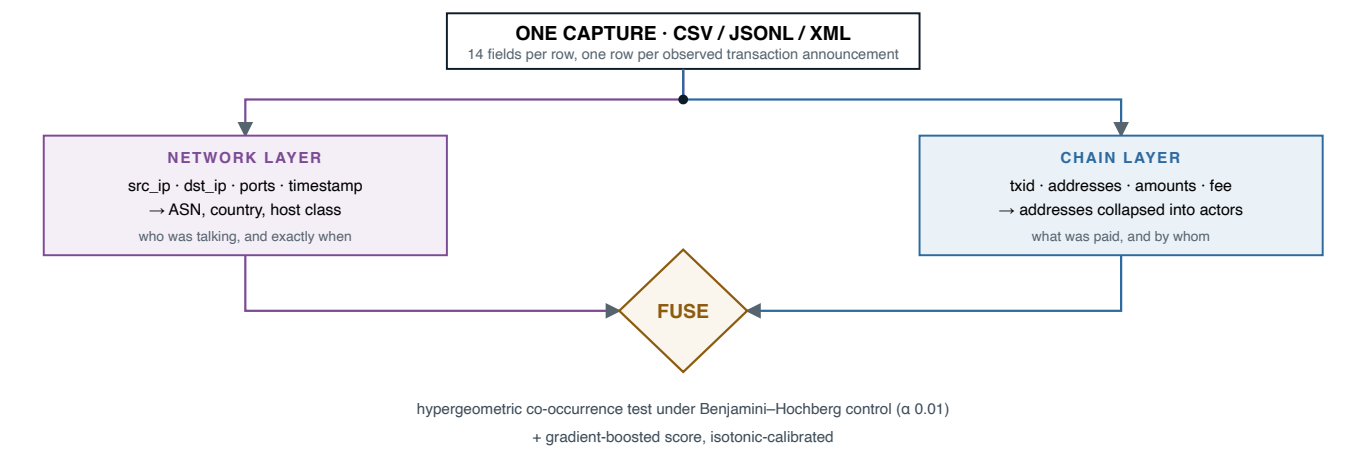


Figure 1. The thesis in one picture. Neither lane can answer the question alone; they meet once, at FUSE. Everything else in this document is how that happens.

The deployment context decides the architecture

The target is an air-gapped analyst workstation. That single constraint explains most of the engineering choices, and each one is a rejection of a more fashionable option:

CHOICE	REJECTED ALTERNATIVE	REASON
DuckDB, a file	PostgreSQL	A service that must be installed, started and kept alive on a machine with no network.
igraph, in memory	Neo4j	No analytical gain at this scale, and a second service to die mid-demo.
FastAPI in the same process as the models	A model-serving sidecar	No serialisation boundary, nothing to orchestrate.
Bundled DB-IP Lite ASN database	A GeoIP web API	Any lookup that leaves the machine breaks the entire premise.
Gradient-boosted trees	Graph neural network	Measured: see §6. SHAP is <i>exact</i> for trees.

The offline claim is enforced, not asserted. make offline-check greps the whole application package for https?:// and fails the build on a match. It cannot tell a citation from an endpoint, so URLs live in documentation and never in code. The container is additionally verified under --network none.

2 · SYSTEM ARCHITECTURE

One process. One container. Nine stages, executed in order, each writing something the next one reads and the provenance receipt can be checked against.

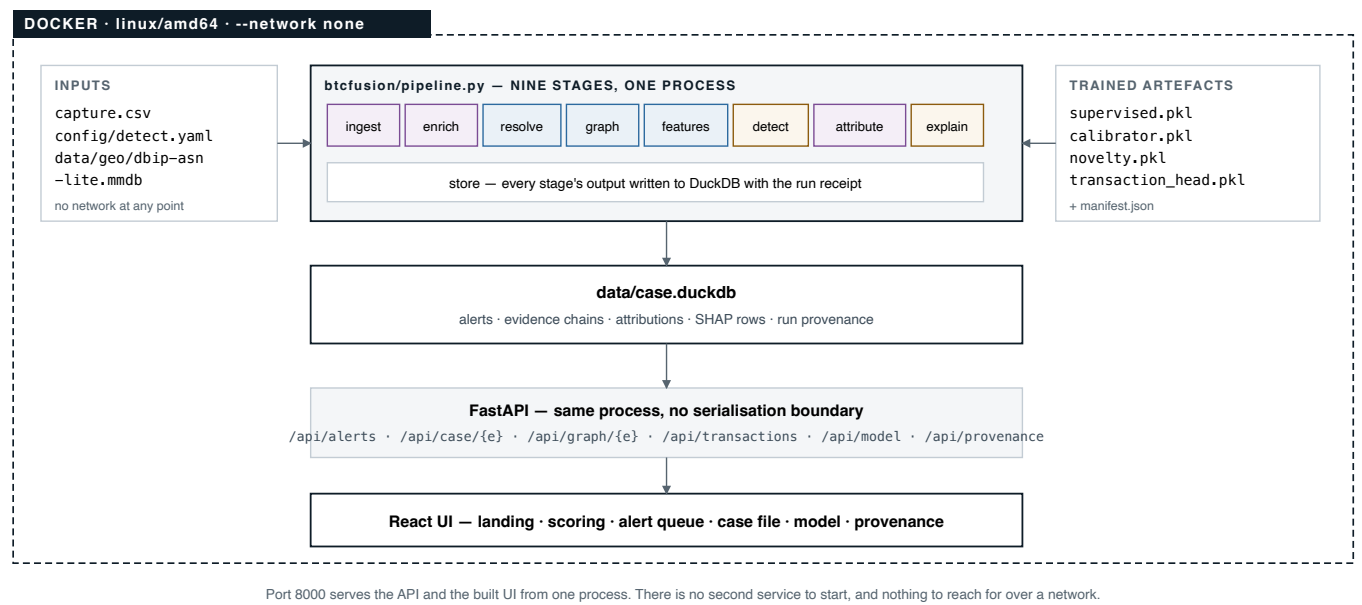


Figure 2. System architecture. Colour marks which layer a stage works in — **network**, **chain**, **model output** — the same palette the interface uses throughout.

Module map

Where each responsibility physically lives, so a reviewer can go from a claim in this document to the code that makes it true.

PACKAGE	RESPONSIBILITY
ingest/	parsers.py (CSV / JSONL / XML → one frame), schema.py, mapper.py, enrich.py (offline GeoIP / ASN)
graph/	resolve.py — common-input ownership; build.py — money graph and the entity × IP matrix
features/	extract.py — 68 engineered features; embeddings.py — the 64-d Node2Vec block
detect/	supervised.py (GBDT), novelty.py (IsolationForest + HDBSCAN), typologies.py (rules, evidence only), transaction_head.py, fuse.py
attribute/	engine.py, diffusion.py, significance.py, infra.py, behaviour.py, confidence.py — one module per step of §8
explain/	narrative.py — SHAP values into English; report.py
eval/	metrics.py, splits.py, labels.py, failures.py, sensitivity.py, external.py (Elliptic), elliptic_pp.py
generator/	The simulator: actors.py, ledger.py, network.py, typologies.py, simulate.py, emit.py, and leak_test.py — the gate of §7
store/ · api/	dao.py — DuckDB access; api/main.py — FastAPI, same process as the models
pipeline.py · train.py	The nine-stage run, and the training entry point that writes artifacts/<profile>/

3 · DATA FLOW — THE NINE STAGES

What each stage consumes, what it computes, and what it leaves behind for the receipt. Timings are the measured bulk-profile run recorded in `manifest.json` → `timings`.

01	INGEST	raw file → one clean frame CSV, JSONL and XML all parse to the same internal frame, and all three are verified to produce identical clean-row counts. Rows failing validation are quarantined with a named reason and counted, never silently dropped — an analyst has to know what was excluded before trusting what was kept. ↳ <code>rows_read</code> · <code>rows_clean</code> · <code>quarantine reasons</code>	3.89 s
02	ENRICH	every IP → ASN, country, host class Lookups run against the DB-IP Lite database bundled inside the image. The host class (residential / mobile / hosting / CDN / VPN / Tor) is what later decides whether an attribution can be trusted or has to be suppressed. ↳ <code>receipt: mmdb:dbip-asn-lite.mmdb</code>	2.36 s
03	RESOLVE	1,810,005 addresses → 829,821 actors Common-input ownership: if one transaction spends from five addresses at once, one party held all five private keys. 635,510 co-spend edges plus 344,938 change edges, transitively closed. Mean cluster purity against ground truth is 0.9956. ↳ <code>mean 2.18 addr/actor</code> · <code>largest 953</code> · <code>688,853 singletons</code>	2.99 s
04	GRAPH	actor-to-actor flow, and entity × IP Two structures, not one: the money graph (PageRank, sampled betweenness) and, separately, the sparse entity × IP co-occurrence matrix the attribution test needs. Sparse rather than dense is what keeps 2.4M rows inside laptop memory. ↳ <code>295,330 graph nodes embedded</code>	0.88 s
05	FEATURES	132 numbers per actor Chain behaviour, timing rhythm, network posture, position in the money graph, plus 64 learned Node2Vec dimensions. Expressed as Polars expressions so the whole matrix computes in parallel rather than row by row. ↳ <code>feature_version 7</code> — the most expensive stage by far	77.77 s
06	DETECT	score every actor, then calibrate it Gradient-boosted trees score all actors; isotonic regression turns the raw score into a probability that means something, so 0.90 is roughly a nine-in-ten chance rather than merely 'higher than 0.80'. Ranking is the classifier's job alone. ↳ <code>fusion weights 1.00 / 0.00 / 0.00</code> · <code>threshold 0.75</code>	*
07	ATTRIBUTE	which IP is really theirs A hypergeometric test per actor–IP pair with Benjamini–Hochberg control at α 0.01, weighted by propagation-tree roots and penalised by infrastructure class. Where the evidence points at shared infrastructure it says nothing rather than guessing. ↳ <code>290,787 pairs tested</code> · <code>28,571 suppressed</code>	*
08	EXPLAIN	why this actor was flagged Exact SHAP — TreeExplainer, not an approximation — for the 60 alerts actually shown. There is no reason to explain 829,821 actors nobody will open. The top contributions become an English narrative an analyst can act on. ↳ <code>shap_top_k 8</code> · <code>explain_top_n 60</code>	*
09	STORE	write the case files and the receipt Alerts, evidence chains with real TXIDs, attributions, SHAP rows and the run provenance all go to DuckDB: the input SHA-256, the seed, the feature version, the model backend and the git commit. ↳ <code>every figure traces to an exact input and an exact model</code>	*

Table 1. The nine stages as executed, with the artefact each contributes to the provenance receipt. Timings are from the bulk training run; detect onward are reported per run rather than per stage.

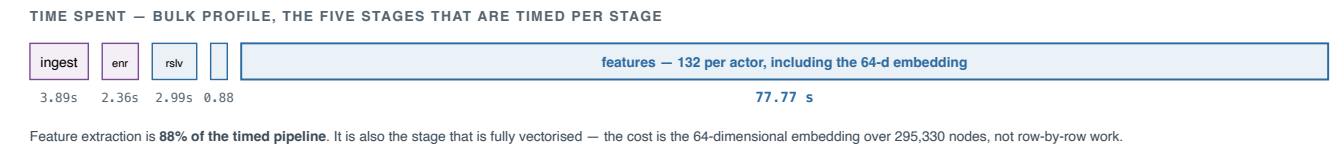


Figure 3. Where the time actually goes, to scale.

Degrade, never fail. If a stage raises, the run continues and the interface reports that component as unavailable. A stack trace in front of an analyst is the one unacceptable outcome, and a partial answer with an honest gap is more useful than no answer at all.

4 • HOW ONE TRANSACTION IS ANALYSED

The stages above describe the machinery. This is what actually happens to a single row of the capture, from the byte that arrives to the sentence an analyst reads.

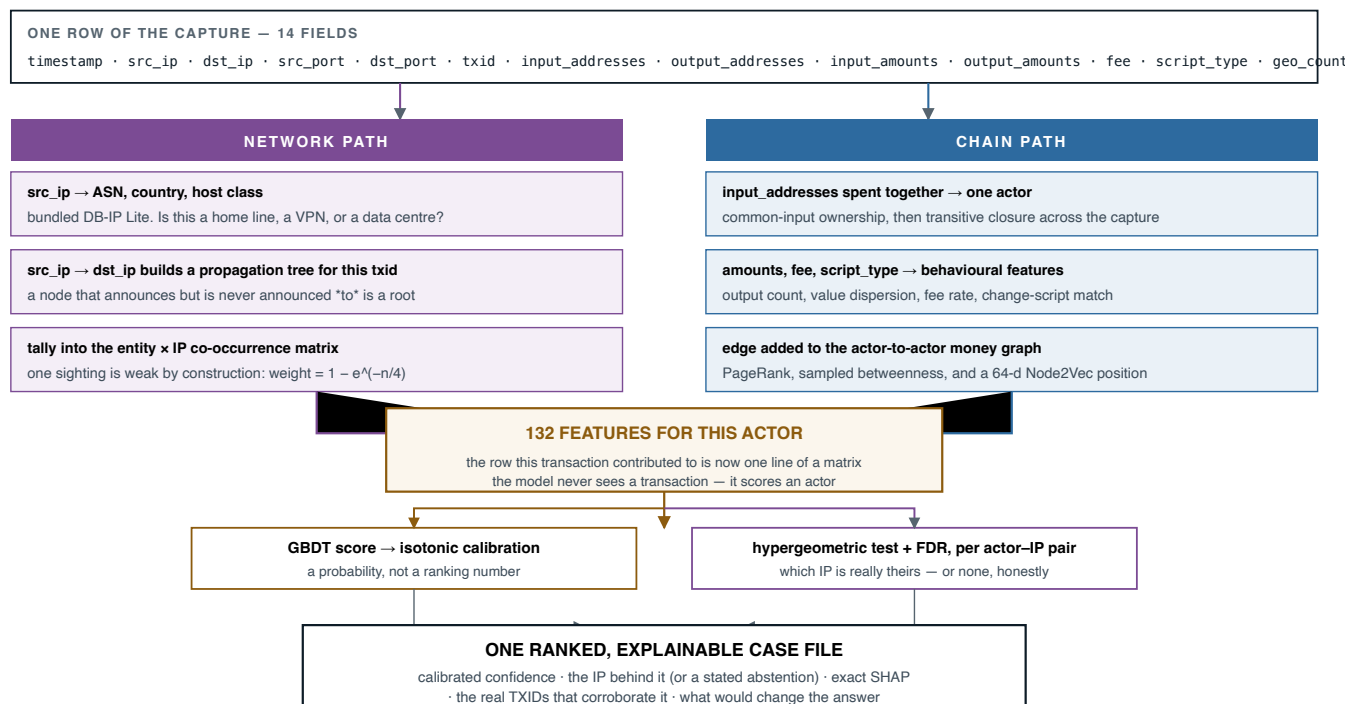


Figure 4. One row's journey. The critical structural point: **the model scores actors, never transactions** — a single transaction is evidence contributing to an actor's feature row. A separate transaction-level head exists for TXID ranking (§10).

Why not score transactions directly? A transaction has almost no behaviour — it has amounts and a timestamp. Laundering is a *pattern over time*: peel chains, rapid layering, structuring. Those are properties of an actor across many transactions, which is why the primary model works at the actor level and reaches PR-AUC 0.3048, while the transaction head reaches 0.0833. Both are reported; neither is hidden.

5 • WHERE THE AI IS

Five learned components, each doing a job that cannot be written as a rule. Every one is trained, versioned and recorded in the manifest — and the ablation in §10 proves that the lift comes from them and not from the rules beside them.

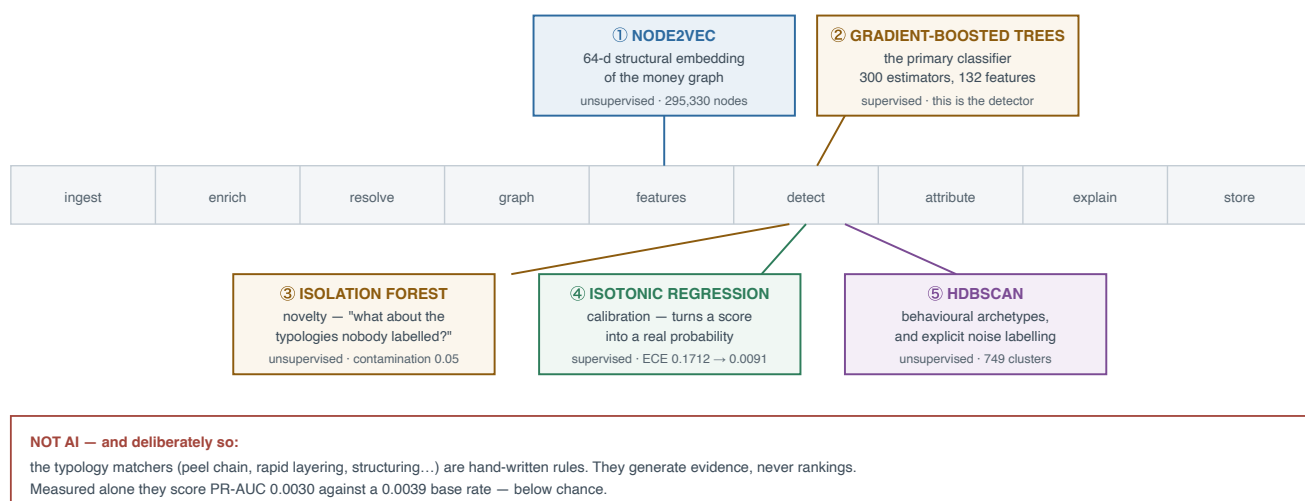


Figure 5. Every learned component, positioned at the stage it runs in. Three of the five are unsupervised, which matters for a domain where labels are scarce and adversaries change behaviour.

Answering the question directly

The AI is not a wrapper around a rules engine, and this is checkable rather than asserted. The ablation below is a row-by-row build-up of the shipped system, and each row is an entry in `metrics.json` → `ablation`:

WHAT IS SWITCHED ON	TEST PR-AUC	× BASE RATE	P@50	READING
Typology rules only	0.0030	0.77×	0.00	Worse than chance. Rules cannot rank.
+ unsupervised novelty	0.0207	5.3×	0.00	Signal appears, but not orderable.
+ GBDT on engineered features	0.2809	72×	0.92	The lift is here.
+ Node2Vec embeddings	0.3231	83×	0.96	Learned topology adds 15% more.
+ calibration (shipped)	0.3048	78×	0.90	4% of ranking bought for honest probabilities.

The base rate on this fold is 0.0039. Rules alone land *below* it. Everything above the base rate is contributed by the learned components — which is precisely what the problem statement's "a *working model*, not just rules" asks to be demonstrated.

A measurement that overturned our own design. The system originally fused three signals into one ranking score at 0.85 supervised / 0.05 novelty / 0.10 evidence. That survived the demo profile and collapsed at a realistic base rate: PR-AUC fell from 0.3231 to **0.0382**, an 88% loss. The evidence term carries no ranking signal at 0.39% positives, and being near-binary it promoted a broad band of unremarkable actors above genuinely suspicious ones. Ranking is now the classifier's job alone — 1.00 / 0.00 / 0.00. Coverage of unlabelled typologies is bought differently: **8 of the 60 queue slots are reserved** for the highest-novelty actors the classifier did not surface. Blending corrupts every rank; reserving costs exactly the slots it uses. Full sweep in `metrics.json` → `fusion_sweep`.

6 · MODEL SPECIFICATIONS

PRIMARY CLASSIFIER

Family	Gradient-boosted decision trees
Reference backend	LightGBM
Shipped backend	sklearn_histgb
Estimators	300
Input features	132
Unit of prediction	actor (entity)
Training rows	177,126
Train duration	596.66 s
Feature version	7
Seed	20260826

The shipped artefacts record sklearn_histgb because they were trained on a macOS host without libomp. A fresh detector inside the Linux container resolves LightGBM — the same histogram algorithm. The manifest records which backend produced every number, so the two are never confused.

NODE2VEC EMBEDDING

Backend	ppmi_svd
Dimensions	64
Walks per node	8
Walk length	16
Return / in-out (p, q)	1, 1
Nodes embedded	295,330

NOVELTY & CLUSTERING

Novelty model	IsolationForest
Contamination	0.05
Archetypes	HDBSCAN
Min cluster size	25
Clusters found	749
Noise fraction	0.539

CALIBRATION & THRESHOLDS

Calibrator	Isotonic regression
Alert threshold	0.75
Max alerts	60
Novelty slots reserved	8
SHAP top-k	8

Why these models, and what was rejected

MODEL	JOB	WHY THIS ONE
Node2Vec (64-d)	Structural roles	Nobody labels "mixer-like topology". The embedding discovers it, and it runs CPU-only.
Gradient-boosted trees	Primary classifier	Best-in-class on tabular data, and SHAP is exact for trees rather than approximated.
IsolationForest	Novelty	Linear-time, no distance assumptions. Answers "what about patterns you never labelled?"
HDBSCAN	Behavioural archetypes	Does not need <i>k</i> , and explicitly labels noise — which is what an outlier is.
Isotonic regression	Calibration	Non-parametric, so it corrects arbitrary miscalibration shapes rather than an assumed one.

Graph neural networks were rejected on evidence, not taste. Weber et al. (2019) found tree ensembles outperformed a GCN on illicit-class F1 on exactly this task. GNNExplainer output is hard to turn into analyst-readable prose, and a GNN is likelier to memorise generator fingerprints — a real risk when the training data is synthetic. The structural signal a GNN would learn is captured instead by the Node2Vec features, whose contribution is measured separately in the ablation (+0.042 PR-AUC).

What the 132 features are

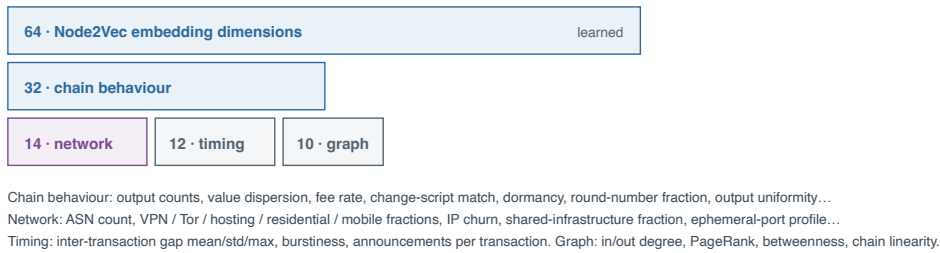


Figure 6. Feature composition. Half the vector is learned rather than engineered — the embedding is the part no analyst could have specified.

THE TEN FEATURES THE MODEL ACTUALLY LEANS ON

FEATURE	GAIN	WHAT IT CAPTURES
max_n_outputs	0.147	The widest fan-out this actor ever produced — the shape of a distribution step.
mean_n_outputs	0.090	Whether fan-out is habitual or a one-off.
degree_in	0.081	How many distinct counterparties pay this actor.
emb_2	0.075	Learned structural position. No hand-written meaning — that is the point.
emb_4	0.068	As above.
in_out_ratio	0.047	Collector versus distributor behaviour.
vpn_frac	0.040	Network layer. Share of announcements from VPN address space.
gap_mean	0.032	Mean interval between transactions — tempo.
tor_frac	0.026	Network layer. Share from Tor exits.
hosting_frac	0.025	Network layer. Share from data-centre ranges.

Three of the top ten come from the network layer — the half of the data a blockchain explorer does not have. That is the correlation the problem statement asks for, appearing as measured feature importance rather than as a claim.

7 • TRAINING DATA AND SPLITS

No dataset was provided, and **no public dataset has a network layer**. So "collection" meant writing a defensible simulator — 1,680 lines, seeded, versioned — and then attacking it harder than a reviewer would.

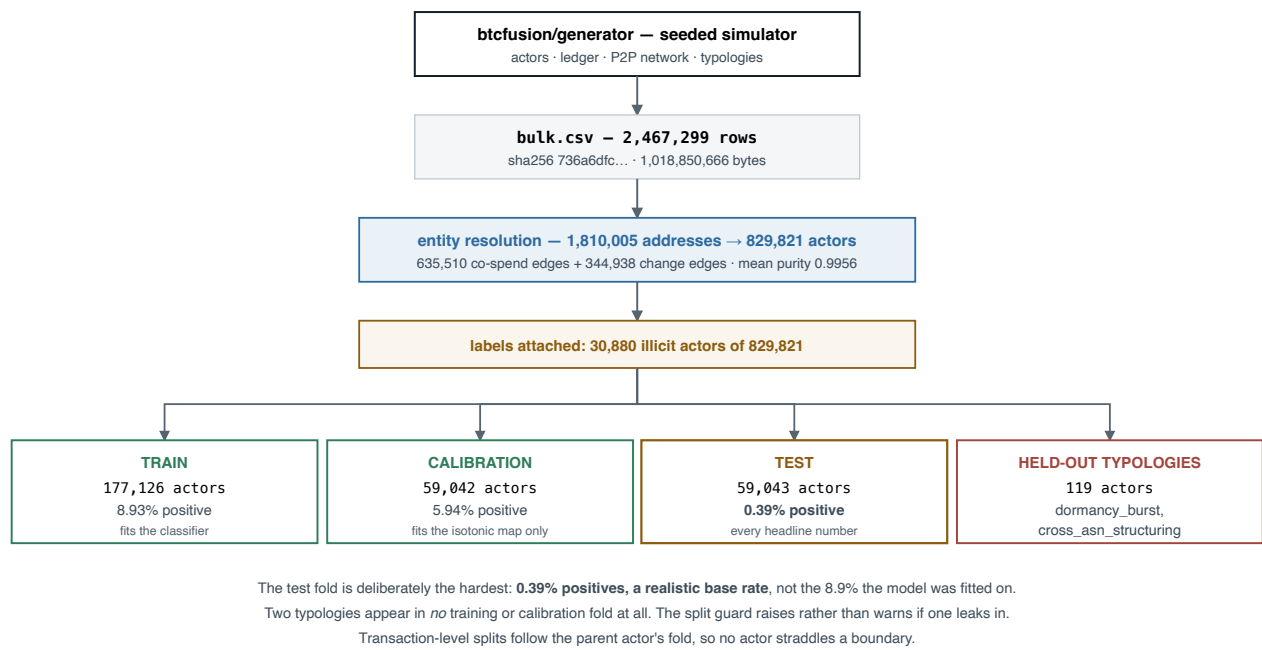


Figure 7. Data generation through to splits. Note the base-rate gradient across the folds — training at 8.93% and testing at 0.39% is deliberate, because the deployment reality is the latter.

The leak test — and the two defects it caught

We wrote the generator, so we know the answers, so every accuracy number is self-referential. A sharp reviewer attacks the data, not the model. So the build gates on a test that trains a classifier *only* on fields with no constructed path to the label, and requires it to score at the base rate.

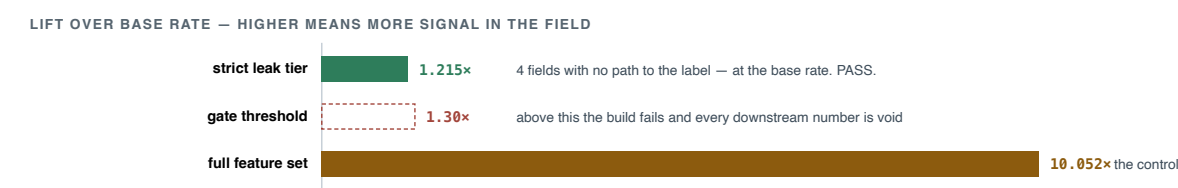


Figure 8. The leak test, from `artifacts/v1/leak_test.json`. The ~8× gap between the strict tier and the control is the evidence that detection comes from topology, timing and network structure rather than from an artefact of how the data was written.

It has earned its keep twice, and both catches are the kind only this test finds:

1. **Crime proceeds were drawn from a different amount distribution than licit payments** — a 1.7× shortcut. Fixed: proceeds now use the same payment-size distribution, with the archetype controlling count and dispersion, never absolute scale.
2. **Mule wallets drew their operating system from a different distribution than everyone else** — 1.2× on ephemeral-port features. The model was reading "no BSD hosts" as evidence of laundering.

Validated against real Bitcoin, not only our own data

Because the synthetic set is ours, the chain-side detector is additionally validated on real labelled Bitcoin. This proves nothing about the network layer — Elliptic has no IP data — and the document says so rather than blurring the two.

VALIDATION	DATA	RESULT	REFERENCE
Elliptic, transaction level	46,564 real labelled transactions, temporal split (train steps 1–34, test 35–49)	F1 0.7595 PR-AUC 0.8009	Weber et al. published 0.79
Elliptic++, actor level	96,023 real addresses, address-disjoint split, 5.56% base rate	PR-AUC 0.3745	the level we actually ship
Clustering heuristic itself	real Bitcoin co-spend clusters	99.79%	label agreement vs 82.09% shuffle control

8 · ATTRIBUTION — THE NETWORK ⇔ CHAIN CORRELATION

The component that answers the problem statement's load-bearing verb. Given an on-chain actor, which network identity is behind it — and how sure are we, honestly?

The problem it has to solve. Naive attribution says "whoever we saw announce it first is the originator". Koshy et al. (FC 2014) and Biryukov et al. (CCS 2014) showed P2P relay patterns leak origin — and Bitcoin Core responded. Modern diffusion applies an **independent randomised delay per peer**, so the first announcement any collector observes is frequently a relay, not the source. First-seen attribution has therefore been wrong since 2015. This engine models the defence instead of ignoring it.

①	entity × IP co-occurrence matrix How often each actor was seen announcing from each IP, counted across every announcement in the capture — not the first one.	engine.py
②	propagation-tree roots, per transaction src_ip → dst_ip edges with timestamps reconstruct who announced to whom. A node that announces but is never announced to within that transaction is a root — materially stronger evidence than “seen first”. Where no candidate survives, the transaction contributes no root evidence at all. Most designs ignore dst_ip entirely and cannot do this.	diffusion.py
③	hypergeometric significance vs a null model A busy relay co-occurs heavily with everyone. The real question is whether the association is stronger than chance given how much traffic that address carries and how many transactions that actor made . Benjamini–Hochberg FDR control is applied across the whole pair set at α 0.01 — testing hundreds of thousands of pairs at an uncorrected $p < 0.01$ would hand back thousands of spurious links.	significance.py
④	infrastructure classification and penalty Two independent sources, because either alone is exploitable: the declared ASN type from the bundled database, and the observed number of distinct actors announcing from that address in this capture.	infra.py
⑤	behavioural timezone inference People keep hours. Which UTC offset makes this actor's activity histogram look most like a human waking, working and sleeping? Derived from behaviour, not from the GeoIP database — so when the two independent methods agree, that is real corroboration. A flat 24/7 histogram fits every offset equally and is reported as such.	behaviour.py
⑥	confidence, interval, counterfactuals The product of everything above. Where every candidate is shared infrastructure, the attribution is suppressed rather than reported at a confidence that would mislead. An attribution engine that always produces an answer is not an attribution engine.	confidence.py

Table 2. The attribution engine, six steps, one module each. The counterfactuals in step ⑥ are computed by re-running this same model with one input changed — so they are a property of the model, not authored text.

confidence =

statistical strength × repeat-observation weight × propagation-root bonus
× infrastructure penalty × timezone corroboration

Repeat-observation weight saturates as $1 - e^{-n/4}$: one sighting earns ~22% of full weight, eight earn ~86%. **One sighting can never on its own produce a confident attribution** — which is exactly the defence randomised diffusion provides.

INFRASTRUCTURE PENALTY

CLASS	MULTIPLIER
residential	1.00
mobile (CGNAT)	0.90
hosting	0.72
CDN	0.30
VPN	0.22
Tor exit	0.12

An ASN labelled residential that carries 400 distinct actors is a shared box whatever the database says — which is why the observed count is weighed alongside the declared class.

MEASURED, BULK PROFILE

Top-1 accuracy	0.9321
Random-choice baseline	0.3378
Top-3 / MRR	0.9328 / 0.9324
Attempt rate	98.94%
Abstentions	624
Mean candidates	3.641
Actor-IP pairs tested	290,787
Significant after FDR	280,910
Propagation-root pairs	80,568
Suppressed	28,571
FDR α	0.01

Abstentions are excluded from accuracy and reported separately. Suppressing an attribution that cannot be made honestly is correct behaviour, not a miss — an engine that always produces an answer is not an attribution engine.

The honest curve: accuracy depends on how much you can see

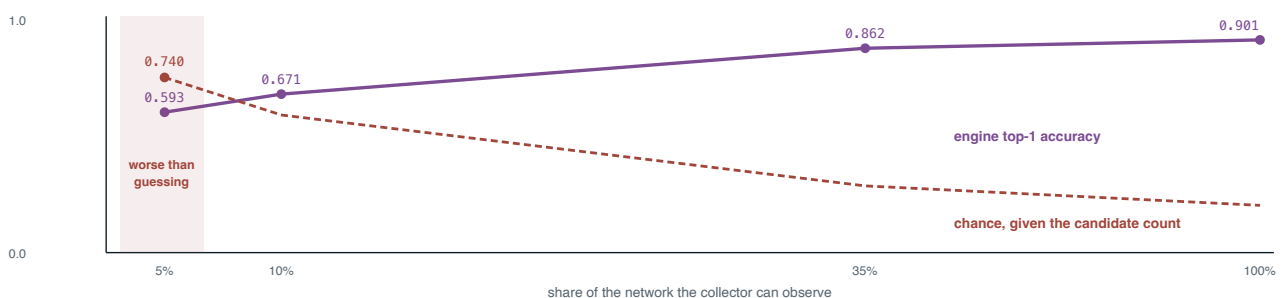


Figure 10. Below roughly 10% observation coverage the engine is worse than guessing — it is mostly seeing relays and picking among them confidently. This is reported rather than hidden: a single accuracy figure at an unstated coverage would be exactly the overclaim this module exists to avoid.

9 • EXPLAINABILITY

Four layers, because "explainable" is a graded deliverable and not garnish. Each answers a different question an analyst actually asks.

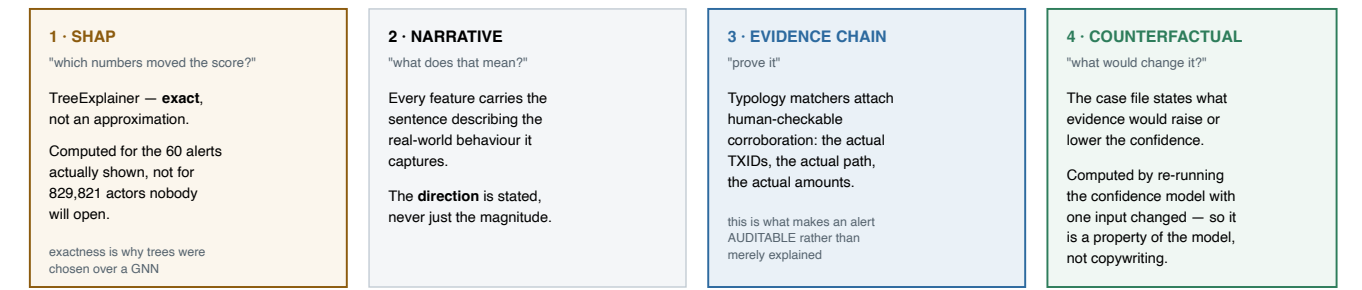


Figure 11. The four explainability layers. Layers 1 and 4 are computed from the model; layer 3 is checkable against the chain itself.

Stated openly: explanation faithfulness is not measured. SHAP values are shown to analysts, but no deletion or insertion test proves the highlighted features actually drive the prediction. This is the one item from the original plan that was never built, and it is recorded as an open item rather than quietly omitted.

10 • MEASURED RESULTS

Everything below is read from `artifacts/bulk/manifest.json`, regenerated by `make reproduce` from the seed. Where a metric can mislead at this base rate, its baseline is printed beside it.

Entity-level detection — the primary model

FOLD	N	POSITIVE RATE	PR-AUC	BASELINE	ROC-AUC	ECE	MCC
train	177,126	0.0893	0.9780	0.0893	0.9963	0.0155	0.8121
calibration	59,042	0.0594	0.8463	0.0594	0.9535	0.0091	0.7909
test	59,043	0.0039	0.3048	0.0039	0.6744	0.0405	0.4017

Read the **test row, not the train row**. PR-AUC 0.3048 against a 0.0039 base rate is **78x chance**. Precision@10 is 0.90, P@25 is 0.96, P@50 is 0.90 — which is what an analyst working a 60-item queue actually experiences.

Accuracy is printed beside its own baseline on purpose. Test accuracy is **0.9960**. An all-negative classifier that flags nothing at all scores **0.9961** — better. That is exactly why accuracy never judges this system, and why MCC and PR-AUC do.

Calibration — does 0.90 mean 90%?

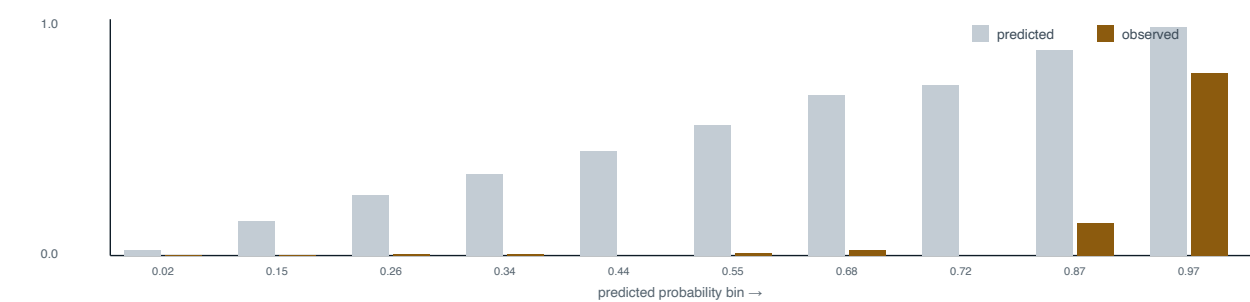


Figure 12. Reliability on the test fold, from `metrics.json` → `reliability_curve`. The top bin predicts 0.9664 and observes 0.7727 across 88 actors. Isotonic regression cut calibration error on the calibration fold from **0.1712 to 0.0091**; on the far harsher test fold it lands at 0.0405. The model is still optimistic in the middle bins, and the chart shows that rather than hiding it.

Transaction-level head

METRIC	VALUE	NOTE
Transactions scored	595,929	2,596 illicit; splits follow the parent actor's fold
Test set	125,308	192 positives — a 0.0015 base rate
PR-AUC	0.0833	vs 0.0015 baseline — 56x chance
ROC-AUC	0.8714	
Precision@10	0.90	the top of the ranked TXID list is usable

Weaker than the actor-level model, and necessarily so — a transaction has amounts and a timestamp, while laundering is a pattern across many transactions. Both are shipped because the problem statement asks for flagged *wallets and transactions*.

Entity resolution and clustering

Addresses	1,810,005	Singleton actors	688,853
Actors recovered	829,821	Mean cluster purity	0.9956
Co-spend edges	635,510	Impure clusters	4,267
Change edges	344,938	Behavioural archetypes	749
Mean addresses / actor	2.18	Noise fraction	0.5387
Largest actor	953 addresses	Illicit actors labelled	30,880

Two levels of clustering, as the problem statement asks: **identity** clustering (addresses → actors, by common-input ownership) and **behavioural** clustering (actors → 749 archetypes, by HDBSCAN). They answer different questions and are never conflated.

11 • LIMITS AND FAILURE MODES

A system that cannot say where it breaks has not been evaluated. These are read from the artefacts, not softened.

Where the model actually fails

ACTOR	KIND	SCORE	WHY
ENT-702409	false positive	0.97	Licit, but its feature profile sits in the overlap between transient behaviour and laundering topology.
ENT-251721	false positive	0.97	Announced almost entirely from shared infrastructure, which lifts network-layer features without the chain layer supporting the flag.
ENT-31606	false negative	0.01	Genuinely rapid_layering, but too few transactions to establish the topology the model needs.
ENT-36685	false negative	0.01	Genuinely peel_chain, same cause: not enough history.

The pattern is consistent and worth stating plainly: **the model needs history**. An actor with a handful of transactions cannot show a topology, so a real criminal who has just started is exactly the case this system is weakest on.

Known limits

LIMIT	WHAT IT MEANS IN PRACTICE
Attribution collapses below ~10% observation coverage	Worse than guessing — see Figure 10. Coverage must be stated with any accuracy claim.
Training data is synthetic	Mitigated by the leak test and by two real-data validations, but the network layer itself has no real-world validation set in existence.
Explanation faithfulness unmeasured	SHAP is shown; no deletion/insertion test proves it drives the prediction.
Scaling is superlinear	4.4× the data cost 8.9× the time. The features stage dominates at 77.77 s of the run.
Calibration is optimistic in mid-range bins	Visible in Figure 12. Trust the ordering more than the absolute number between 0.3 and 0.7.
Transaction head is weak in absolute terms	PR-AUC 0.0833. Usable at the top of the list (P@10 0.90), not as a filter.

What makes every number in this document checkable

Each run writes a provenance receipt: the input file's SHA-256, the seed, the feature version, the model backend and the git commit. The trained artefacts carry the same. The container build additionally re-derives the host's score fingerprint inside Linux and **fails the build if it drifts** — because the container installs scikit-learn 1.7.2 while the shipped artefacts were pickled by 1.9.0, and "the scores are bit-identical" is a fact about two specific versions rather than a property of the design.

Input	bulk.csv • sha256 736a6dfc3026db82...
Size	1,018,850,666 bytes
Seed	20260826
Feature version	7

Reproducing this document's figures

COMMAND	WHAT IT REGENERATES
<code>make reproduce</code>	Every number in §5, §6, §7, §8 and §10 from the seed.
<code>make verify-all</code>	The machine-checkable subset of the compliance matrix in one pass.
<code>make leak-test</code>	Figure 8 — and gates the build at 1.30×.
<code>make validate-external</code>	Elliptic, 46,564 real transactions.
<code>make validate-elliptic-pp</code>	Elliptic++, 96,023 real addresses.
<code>make docker-build && make docker-verify</code>	Five offline steps under <code>--network none</code> on linux/amd64.
<code>make offline-check</code>	Fails on any URL in application code.

BTC-Fusion — SIH 2026 · PS 26146 · NTRO · Blockchain & Cybersecurity. Deeper treatments live beside the code: `docs/technical_writeup.md` (approach, model choice, evaluation), `docs/attribution_method.md` (the statistics), `docs/generator_parameters.md` (every synthetic-data parameter and its justification), `docs/model_card.md` (intended use and failure modes), `docs/external_validation.md` (what each validation does *not* prove).